PortSwigger Lab - Unprotected Admin Functionality

Platform: PortSwigger Web Security Academy
Category: Access Control Vulnerabilities
Lab Link: https://portswigger.net/web-security/access-control/lab-unprotected-admin-functionality

1. Objective

Understand how improperly protected administrative functionality can allow unauthorized users to access privileged operations. Learn how publicly accessible files, such as `robots.txt`, can reveal hidden application paths and how missing server-side authorization checks can lead to vertical privilege escalation.

2. Key Concepts Learned

- **Access Control:** Security mechanisms that determine which users are allowed to access resources or perform specific actions.
- **Unprotected Functionality:** Occurs when sensitive application features are accessible without proper authorization checks.
- **Vertical Privilege Escalation:** Gaining access to functionality intended for users with higher privileges, such as administrators.
- **Administrative Panel Discovery:** Identifying sensitive application paths that are not linked in the normal user interface.
- **robots.txt:** A publicly accessible file that provides instructions to search engine crawlers about which paths should not be crawled. It can unintentionally reveal sensitive application paths.
- **Security by Obscurity:** Relying on hidden or difficult-to-guess URLs instead of implementing proper security controls. Hiding an admin URL does not protect the functionality.
- **Server-Side Authorization:** Verifying permissions on the server before allowing a user to perform a sensitive operation.
- **Least Privilege Principle:** Giving users only the permissions required for their roles and responsibilities.

3. Tools and Websites Covered

- **PortSwigger Web Security Academy:** Provided the intentionally vulnerable lab environment and practical exercise.
- **Web Browser:** Used to access the lab, inspect `robots.txt`, navigate to the disclosed administrative path, and interact with the application.
- **robots.txt:** Used as an information source to discover the administrative panel's location.

Note: Burp Suite is not required for the documented solution to this particular lab.

4. Practical Work

- Opened the PortSwigger lab and accessed its web application.
- Navigated to the `/robots.txt` path to inspect the instructions provided to search engine crawlers.
- Examined the `Disallow` entry and identified the administrative panel path disclosed in the file.
- Replaced the `/robots.txt` path with `/administrator-panel` in the browser URL.
- Accessed the administrative panel and examined the available functionality.
- Used the panel's user-management functionality to delete the specified user, `carlos`.
- Confirmed successful lab completion through the PortSwigger interface.

Useful URL Paths

```
/robots.txt
/administrator-panel
```

5. Important Security Observations

- **A hidden URL is not an access-control mechanism.** Administrative functionality must remain protected even when someone knows its exact URL.
- **robots.txt is publicly accessible.** Its `Disallow` entries are instructions for crawlers, not security restrictions for visitors.
- **The real vulnerability was missing authorization.** Discovering the admin path alone did not create the vulnerability; unauthorized access to the administrative functionality did.
- **Administrative endpoints require independent protection.** Restricting access to an admin link in the user interface is insufficient if the endpoint itself remains accessible.
- **Sensitive functionality should be protected on the server side.** Access decisions must not depend only on whether a user can see a particular link or page.
- **Unauthorized administrative access can have serious consequences.** An exposed panel may allow account deletion, modification of user information, or other privileged operations.
- **Unlinked endpoints are still part of the attack surface.** Security testing should examine application paths beyond those visible through normal navigation.

6. Security Recommendations

- Implement server-side role-based access control for administrative functionality.
- Verify the authenticated user's role and permissions before executing every privileged operation.
- Apply a deny-by-default approach to sensitive endpoints.

- Do not rely on hidden URLs, robots.txt restrictions, or interface-level controls for security.
- Restrict administrative functionality to authorized accounts and trusted access paths.
- Maintain logs of sensitive administrative actions and investigate unauthorized access attempts.
- Regularly test administrative endpoints using both ordinary-user and administrator accounts.

7. Key Learnings

This lab demonstrated how a small information disclosure can help reveal a much more serious access-control weakness. I learned that `robots.txt` can expose paths that developers may not intend ordinary users to discover, but the main security failure occurs when the application does not enforce authorization on the administrative endpoint.

The most important takeaway was the difference between **hiding functionality and securing functionality**. A URL may be difficult to discover, but once someone finds it, the application must still verify their permissions. This reinforced the importance of server-side authorization, least privilege, and testing sensitive endpoints directly rather than relying only on the application's visible navigation.

Status: Complete